TASK 4 – Use Macros Safely 4 Marks

Bloom's Level: K4 – Analyse

Write macros for small repeated calculations (for example: converting rupees to paise, finding the larger of two values).

Discuss:

• How a macro is expanded by the preprocessor, shown using gcc -E

• Why SQUARE(x) written as x*x gives a wrong result for SQUARE(a+1), and how parentheses fix it

• Why MAX(a, b) can evaluate an argument twice (for example MAX(i++, j))

• When a function is a better choice than a macro Alternative Concept and Complexity Analysis If the MAX and SQUARE macros were replaced by static inline functions, what would the solution be? Analyse the time cost of a call versus an expanded macro, and the code size (space) when the macro is used k times in the program, since a macro is copied at every use. Evidence Expected Correct, fully parenthesised macros with gcc -E evidence and an explanation of each macro pitfall

TASK 4 – Use Macros Safely

Bloom's Level: K4 – Analyse
Marks: 4

1. Definition of Macros

A macro is defined using the #define preprocessor directive in C. The preprocessor replaces the macro name with its corresponding code before the compiler compiles the program. Macros can reduce repeated code, but they must be written carefully.

Examples of safe macros:

#define RUPEES_TO_PAISE(r) ((r) * 100)

#define SQUARE(x) ((x) * (x))

#define MAX(a, b) (((a) > (b)) ? (a) : (b))

Parentheses around each argument and the entire expression help prevent errors caused by operator precedence.

2. How Macro Expansion Works

The C preprocessor expands macros before compilation. The command gcc -E displays the preprocessed source code.

For example:

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))


int main(void)

{

   printf("%d\n", SQUARE(5));

   return 0;

}
```

Save this program as task04.c. Run the following command in Command Prompt:

```
gcc -E task04.c -o task04_preprocessed.c
```

Open task04_preprocessed.c and search for the expanded expression. The relevant part will contain:

```
printf("%d\n", ((5) * (5)));
```

This demonstrates that the macro is expanded before compilation.

3. Why Parentheses Are Important in SQUARE

An unsafe macro is:

```
#define SQUARE(x) x * x
```

When called as SQUARE(a + 1), it expands to:

```
a + 1 * a + 1
```

Because multiplication has higher precedence than addition, the expression does not calculate the square of a + 1.

The safe macro is:

```
#define SQUARE(x) ((x) * (x))
```

It expands to:

```
((a + 1) * (a + 1))
```

Therefore, the expression is evaluated correctly.

4. Why MAX Can Evaluate an Argument Twice

Consider this macro:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

If the first argument is i++, the argument can be evaluated more than once. For example, in MAX(i++, j), the increment may occur during the comparison and again when the first argument is selected.

This makes the result difficult to predict and can modify i unexpectedly.

Avoid side effects in macro arguments. Use ordinary variables or a function when arguments involve operations such as i++.

5. When a Function Is Better Than a Macro

A function is preferable when:

- Arguments may have side effects.
- Type checking is important.
- The calculation is complex.
- Debugging and maintenance should be easier.
- The calculation should not be textually duplicated at every use.

Macros are useful for simple, repeated expressions, but functions are generally safer for more complex calculations.

6. Alternative Concept: Static Inline Functions

Macros can be replaced with static inline functions:

```
static inline int square(int x)
{
   return x * x;
}


static inline int maximum(int a, int b)
{
   return (a > b) ? a : b;
}
```

Example usage:

```
int result1 = square(5);
```

int result2 = maximum(10, 20);

These functions evaluate each argument once, provide type checking, and avoid the repeated-evaluation problem of macros. An inline function may be expanded by the compiler or implemented as a normal function call; the compiler decides based on optimization and other factors.

7. Time and Space Complexity Analysis

Factor	**Macro**	**Static inline function**
Calculation time	Usually (O(1)) for these examples	Usually (O(1)) for these examples
Call overhead	No function-call overhead from macro expansion	May have no call overhead if inlined; otherwise a call may add overhead
Type checking	No inherent function-parameter type checking	Provides normal type checking
Argument evaluation	May evaluate arguments multiple times	Each argument is evaluated once for a normal function call
Code size when used (k) times	Expanded code may be duplicated at every use	Compiler may inline, share or otherwise optimize the implementation

For a simple square or maximum calculation, the time complexity is (O(1)). If a macro's expanded body is copied at (k) locations, its generated code can grow with (k), so the space cost of the repeated code may be (O(k)). An inline function does not guarantee smaller code because the compiler may also inline it at multiple call sites.

Code :

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((r) * 100)

#define SQUARE(x) ((x) * (x))

#define MAX(a, b) (((a) > (b)) ? (a) : (b))

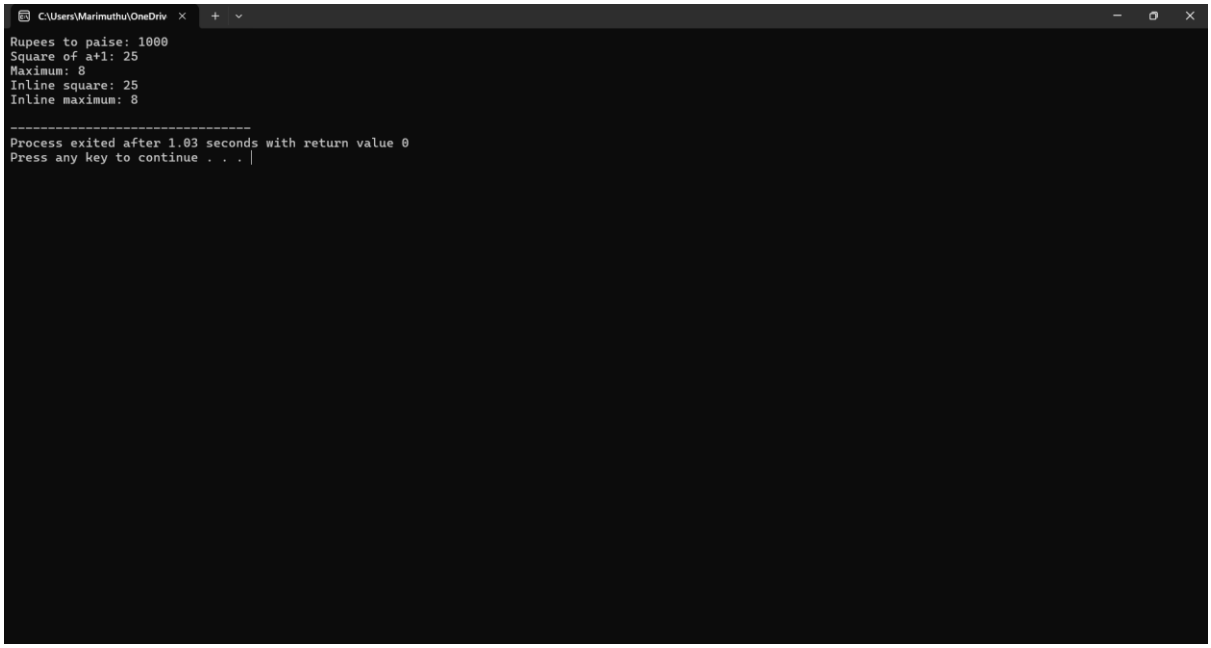

static inline int square(int x)

{
```

```
    return x * x;
}
static inline int maximum(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 4;
    int i = 5;
    int j = 8;
    printf("Rupees to paise: %d\n", RUPEES_TO_PAISE(10));
    printf("Square of a+1: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(i, j));
    printf("Inline square: %d\n", square(a + 1));
    printf("Inline maximum: %d\n", maximum(i, j));
    return 0;
}
```

Output:



8. Conclusion

Macros are useful for small repeated calculations, but missing parentheses can cause incorrect results, and arguments with side effects can be evaluated multiple times. Fully parenthesised macros improve correctness. Static inline functions are often safer because they provide type checking and evaluate each argument once. The gcc -E command provides evidence of how macro expansion works before compilation.